PASSWORD STRENGTH CHECKER

Project 1 — Cyber Security

DecodeLabs Industrial Training Kit | Batch 2026

Project Name	Password Strength Checker
Track	Cyber Security - Defensive Logic
Project Number	Project 1 of the Internship Series
Language	Python 3
GUI Library	Tkinter (built-in, no install needed)
Organization	DecodeLabs
Batch	2026
Project made by	Dinesh Lal Das

1. Project Overview

This project is the first milestone of the DecodeLabs Cyber Security internship. The goal was to build a program that checks whether a password is Weak, Medium, or Strong based on a set of security criteria.

The project focuses on the defensive side of cybersecurity. Before learning how to break into systems, it is important to understand what makes data secure in the first place. A password is the first line of defense for any user account, so evaluating password strength is a fundamental security skill.

The program was built using Python with a simple desktop GUI made using Tkinter. No external libraries were needed since Tkinter comes bundled with Python.

---

2. How I Built It

I started by planning the logic on paper first. I asked myself: what makes a password strong? Based on what I learned, a password needs to be long enough and have a good mix of different character types.

2.1 Planning the logic

I decided the program would check 6 things and give 1 point for each:

- Is the password at least 8 characters long?
- Is it 12 or more characters? (extra point for being longer)
- Does it have at least one uppercase letter?
- Does it have at least one lowercase letter?
- Does it have at least one number?
- Does it have at least one symbol like ! @ # $?

After scoring, the result is:

- Score 0-2 → WEAK (shown in red)
- Score 3-4 → MEDIUM (shown in orange)
- Score 5-6 → STRONG (shown in green)

2.2 Building the GUI

I used Tkinter to make a desktop window. I chose a dark background color (#1e1e1e) because it looks more like a security tool. The layout is simple: an input field at the top, a button to check, a color bar that fills up based on the score, and a tips section at the bottom.

The show/hide button next to the input field toggles the password visibility, which is a common feature in any login form.

---

3. Key Functions Explained

3.1 check_password(password)

This is the main logic function. It takes the password string as input and returns two things: the strength label and the score number.

How it works step by step:

- First checks if the password is less than 8 characters. If yes, returns WEAK immediately with score 0. No point checking anything else.
- Then loops through every character in the password using a for loop to set four boolean flags: has_upper, has_lower, has_num, has_symbol.
- After the loop, adds points to the score based on which flags are True and whether length conditions are met.
- Uses if/elif to decide the final strength label from the score.

Code snippet:

```
def check_password(password):
    if len(password) < 8:
        return 'WEAK', 0
    score = 0
    has_upper = False
    for c in password:
        if c.isupper():
            has_upper = True
    if has_upper:
        score += 1
    ...
    return strength, score
```

3.2 get_tips(password)

This function generates a list of suggestions to show the user. It checks each condition separately and appends a tip if that condition is not met.

This function uses the any() method which I learned is a cleaner way to check if at least one character in the string satisfies a condition:

```
has_upper = any(c.isupper() for c in password)
```

This reads: for each character c in the password, check if it is uppercase. If any one of them is, has_upper becomes True.

3.3 check_btn_clicked()

This is the button handler function. It runs when the user clicks the Check Password button. It:

- Gets the text from the entry field using entry.get()
- Calls check_password() to get the strength and score
- Updates the result label text and color
- Changes the width of the bar frame based on score (each point = 60px)
- Calls get_tips() and displays the tips in the label

3.4 toggle_password()

A small helper function for the show/hide button. It checks what the entry field's show property is currently set to. If it is showing asterisks (*), it removes the masking. If it is showing plaintext, it puts the masking back.

```
def toggle_password():
    if entry.cget('show') == '*':
        entry.config(show='')
        show_btn.config(text='hide')
    else:
        entry.config(show='*')
        show_btn.config(text='show')
```

4. Key Python Concepts Used

Concept	How it was used
String methods	.isupper(), .islower(), .isdigit() to check character types
for loop	Looping through each character in the password string
Boolean flags	has_upper, has_lower, has_num, has_symbol to track what was found
if / elif / else	Deciding the strength label from the score
any()	Cleaner way to check if any character meets a condition
string.punctuation	Built-in constant with all symbol characters
Functions	Separated logic (check_password), tips (get_tips), and UI actions
Tkinter	Built the desktop window, labels, buttons, entry, and frames
len()	Checking password length for the 8 and 12 character conditions
list.append()	Building the tips list by adding messages one by one

5. Why Password Strength Matters

According to security research, the majority of hacking-related data breaches happen because of weak or stolen passwords. When a password is short or uses only letters, attackers can guess it quickly using automated tools called brute force attacks.

A password that is 6 characters long using only lowercase letters has about 300 million possible combinations. A modern computer can crack that in seconds. But a 12-character password with uppercase, numbers, and symbols has trillions of combinations, making it practically impossible to brute force.

This is why the program checks for all these conditions and not just length alone.

---

6. How to Run the Program

Requirements: Python 3 installed on your machine. No extra packages needed.

Run command:

```
paswword strength checker/password.py
```

The app will open as a desktop window. Type a password in the input box and click Check Password to see the result.

---

7. What I Learned

- How to break a problem into small functions instead of writing everything in one place
- String handling in Python - how to loop through characters and check their type
- The logic behind password security and why different character types matter
- How to build a basic desktop UI using Tkinter
- The difference between checking conditions with a for loop vs using any()
- Why validation is important as a first step before any other security process

---

8. Possible Improvements

If I had more time or wanted to extend this project, I would add:

- A check for common weak passwords like 'password123' or '12345678'
- A password generator button that creates a strong password automatically
- A history feature that remembers which passwords were already checked
- Color animation on the strength bar when the score changes